

Homework 01

*Due: August 8, 2026 at 11:59 PT**August 4, 2026*

Sources

- Book: <https://dsc40b.com/materials/default/notes/book.pdf>
- Lecture 01: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec01/Lecture_01_blank.pdf
- Lecture 02: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec02/Lecture_02_blank.pdf
- Lecture 03: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec03/Lecture_03_blank.pdf

Problem 1

Loop Invariants.

Consider the following algorithm:

```
1 def second_largest(arr):  
2     n = len(arr)  
3     largest = max(arr[0], arr[1])  
4     second = min(arr[0], arr[1])  
5     for i in range(2, n):  
6         if arr[i] > largest:  
7             second = largest  
8             largest = arr[i]  
9         elif arr[i] > second:  
10            second = arr[i]  
11     return second
```

Assume $n \geq 2$ and that `arr` contains no duplicate maximum value.

- (a) **[10 points]** State a single loop invariant, in terms of `largest` and `second` jointly, that holds after the α^{th} iteration of the loop.
- (b) **[10 points]** Prove the invariant using the four-step scaffold from lecture: base case, inductive hypothesis, inductive step, and finalize.
- (c) **[5 points]** Use the invariant at loop termination to conclude that the algorithm is correct.

Problem 2

Time-per-execution & Θ Derivation.

(a) [15 points] Consider:

```
1 def count_pairs(arr):
2     n = len(arr)
3     count = 0
4     for i in range(n):
5         for j in range(i + 1, n):
6             if arr[i] == arr[j]:
7                 count += 1
8     return count
```

Build a time-per-execution / number-of-executions table for every line, derive $T(n)$, and give $\Theta(T(n))$.

(b) [15 points] Consider:

```
1 def scan_fixed(arr):
2     n = len(arr)
3     total = 0
4     for x in arr:
5         for k in range(10):
6             total += x * k
7     return total
```

Repeat the same analysis: build the table, derive $T(n)$, and give $\Theta(T(n))$. In one or two sentences, explain why this is *not* $\Theta(n^2)$ despite the nested loop.

You may want to refer to the arithmetic series covered in Lecture 03.

Problem 3

Formal Asymptotics & Case Analysis.

- (a) [15 points] Order the following functions by asymptotic dominance. That is, give an ordering $f_1, f_2, \dots$ such that for all $i \geq 1$, $f_i(n) = O(f_{i+1}(n))$.

$$2^n, \quad 1000n, \quad n(n-1)/2, \quad n^2, \quad 2^{2n}, \quad n!, \quad n^3 - 50n + 42, \quad 5^4$$

- (b) Using the formal definition discussed during Lecture 03,

[7.5 points] Prove $2^{n+1} = O(2^n)$.

[7.5 points] Disprove $2^{2n} = O(2^n)$.

- (c) [15 points] Consider linear search:

```
1 def linear_search(arr, target):  
2     for i in range(len(arr)):  
3         if arr[i] == target:  
4             return i  
5     return -1
```

Give the best-case, worst-case, and average-case running time in Θ -notation, assuming `target` is present in `arr` and is equally likely to be at any index. Then, in one sentence, explain why “ O = worst case” is *not* a correct general statement.

Consider that for average-case, your runtime assesses the probability that the target can be any element chosen at random.